

AI Trip Planner

Complete Project Guide

Built, Debugged & Deployed — LLMs, LangChain, LangGraph, Streamlit

This guide covers, in simple words: the full Week 1-4 build journey, every real bug and issue found and fixed (including live production issues after deployment), how the project works step by step, the prompt customizations added, and the complete GitHub + Streamlit Community Cloud deployment process - plus tips, resume notes, and interview talking points.

Project Type	Agentic AI Travel Assistant (tool-using LLM agent)
Core Framework	LangChain + LangGraph
Interface	Streamlit (calls the agent directly - cloud-ready, single process)
LLM Provider	Groq (free) - llama-3.3-70b-versatile
Status	Live and deployed on Streamlit Community Cloud
Cost	100% free to build and run (no paid APIs required)

Project Report, Debug Log & Deployment Guide

Contents

1. Introduction - Why This Project Exists
2. Project Workflow (Current Deployed Architecture)
3. Week 1 - Foundations & Setup
4. Week 2 - Building the Tools
5. Week 3 - LangGraph Agent Workflow
6. Week 4 - API, UI & Integration
7. All Bugs & Issues Found and Fixed
8. Tools & Models Used (Step by Step Flow Diagram)
9. Prompt Customizations Added
10. Deployment: GitHub + Streamlit Community Cloud
11. How to Run the Project (Local & Live)
12. Tips
13. Resume Notes
14. Interview Talking Points

1. Introduction - Why This Project Exists

A normal AI chatbot only knows what it learned during training - it has no live internet access. It does not know today's weather, today's currency rates, or which restaurants are open right now. If you ask it directly, it either admits it does not know, or worse, confidently makes up a wrong answer (hallucination). For a trip planner, a wrong weather guess or a made-up hotel price is genuinely harmful, not just annoying.

The fix used in this project: give the AI model **tools** - small functions it can call mid-conversation to fetch real, live data - and let it decide, turn by turn, whether it needs to use one before answering. This is what turns a plain chatbot into an **agent**. This project, AI Trip Planner, is now fully built, debugged through several real production issues, and **deployed live** on Streamlit Community Cloud - a complete journey from first line of code to a working public app.

What the finished, deployed project does

- Takes a plain request like *'Plan a trip to Goa for 5 days'*, respecting the exact trip length requested
- Decides on its own which tools it needs to call to answer correctly
- Fetches live weather, live places/attractions, live currency rates, and calculates a real budget
- Returns TWO complete plans (popular/tourist and off-beat/local) with descriptions and map links
- Displays the budget as a proper Markdown table, and live weather compared against the seasonal average
- Automatically retries itself if a model glitch occurs mid-response
- Runs 100% free, and is live for anyone to use at a public URL

2. Project Workflow (Current Deployed Architecture)

This is the CURRENT architecture, as actually deployed - not the original design. Early on, Streamlit talked to a separate FastAPI backend over HTTP (two processes, two terminals). That works fine locally, but Streamlit Community Cloud can only run one process, so this was changed: Streamlit now calls the LangGraph agent directly, in the same process. The diagram below reflects this real, working design.

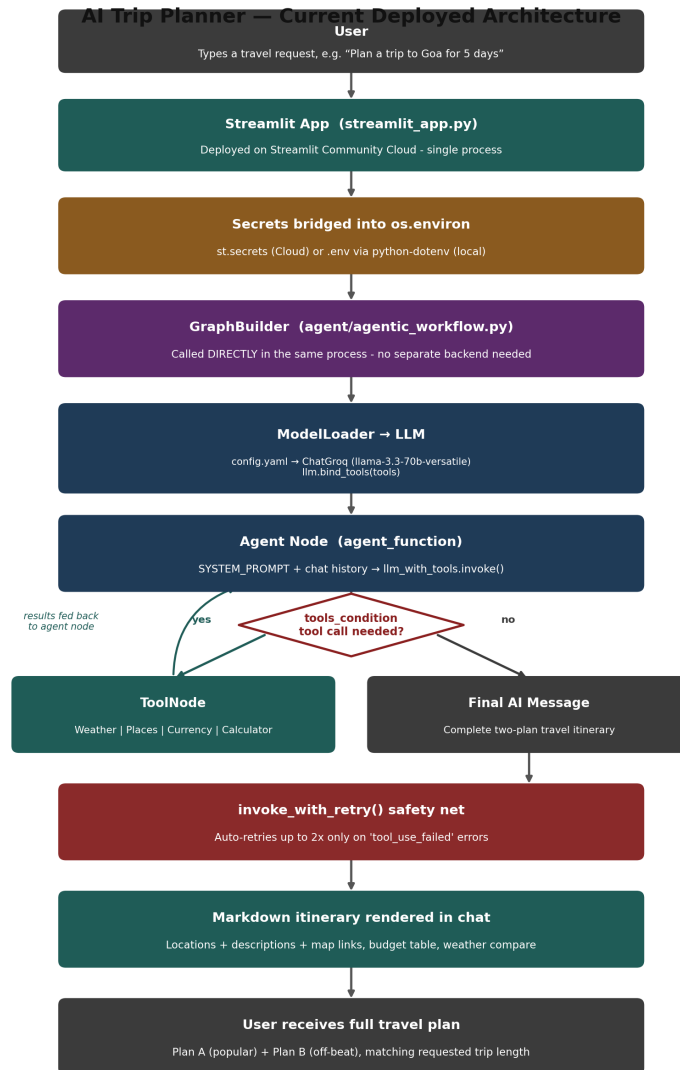


Figure 1: Current deployed request flow of the AI Trip Planner application.

3. Week 1 - Foundations & Setup

WEEK 1 — Foundations & Setup

Week 1 is about getting the project skeleton right before writing any AI logic: a clean Python environment, an organized folder structure, centralized settings, and safe handling of secrets.

Environment setup: Python 3.10 + uv

Pinned to Python 3.10, set up with uv instead of classic pip/venv for speed and simplicity. A virtual environment keeps this project's packages isolated from every other project on the computer.

Project folder structure

Split into agent/, tools/, utils/, prompt_library/, config/, plus main.py and streamlit_app.py - each folder with one clear job, so bugs are easy to find and pieces are easy to swap out later.

Configuration file (config.yaml)

The model name lives in exactly one place. Change it once, everything that reads it follows automatically.

Secrets (.env file)

API keys never typed directly into code - read from .env locally via python-dotenv, keeping them out of version control entirely.

Free-only decision

Deliberately built around free-tier APIs only (Groq, Tavily, OpenWeatherMap, Exchange Rate API), with Google Places treated as optional since it requires Google Cloud billing.

4. Week 2 - Building the Tools

WEEK 2 — Building the Tools

Week 2 gives the AI its 'hands' - independent, tested tools that each fetch one kind of real-world information.

The @tool decorator

Bridges a normal Python function to the AI model, using its docstring so the model knows when and how to call it.

Two layers per tool

A plain utils/ class does the real API work; a thin tools/ wrapper exposes it as an @tool function - reusable and independently testable.

The four tools built

- • **Weather Info Tool** - live current + forecast data from OpenWeatherMap
- • **Place Search Tool** - Google Places first, Tavily as a free automatic fallback
- • **Currency Converter Tool** - live conversion rates from Exchange Rate API
- • **Expense Calculator Tool** - guaranteed-correct budget maths in plain Python

Why a calculator tool at all

LLMs predict likely text, they don't truly calculate - a dedicated tool guarantees the arithmetic is always correct, which matters for a budget feature.

Testing before connecting to the AI

Every tool tried by hand in a notebook first, so problems are caught in a 5-line test instead of a 500-line agent trace.

5. Week 3 - LangGraph Agent Workflow

WEEK 3 — LangGraph Agent Workflow

Week 3 connects the tools to the LLM and turns it into an actual decision-making agent using LangGraph.

ModelLoader

Reads config.yaml and returns the correct LLM object - one place decides, everything else just asks for 'the LLM'.

The system prompt

Explicit instructions on structure, so the AI gives a consistent, reliable format every time instead of a different structure on every request.

bind_tools()

The line that actually teaches the model which functions it can call, what each does, and what input each needs.

Why LangGraph, not a simple script

A real request doesn't clearly say which tools are needed. LangGraph lets the AI decide for itself, in a loop, instead of hardcoding every situation in advance.

StateGraph and the agent/tools loop

An agent node runs the LLM; a tools node executes whatever was requested; a conditional edge decides whether to loop back or finish.

Visualizing the graph

A real, auto-generated picture of the compiled graph - wiring mistakes are obvious in a picture, easy to miss in code.

6. Week 4 - API, UI & Integration

WEEK 4 — API, UI & Integration

Week 4 turns the working agent into an actual usable app - and this section now reflects the REAL, final design after a deployment-driven architecture change.

FastAPI backend (still available)

main.py still exposes a POST /query endpoint - useful for direct API testing or a future different frontend - but is no longer required for the deployed chat UI to work.

Streamlit frontend - now calls the agent directly

Originally, streamlit_app.py sent an HTTP request to a separately-running FastAPI server. This breaks on Streamlit Community Cloud, which only runs one process. It was rewritten to import and invoke the LangGraph agent directly in the same process - works identically locally and in the cloud, and no longer needs a second terminal.

Secrets bridging for the cloud

Streamlit Cloud provides keys via st.secrets, not a .env file. The app checks st.secrets first (wrapped safely, since it errors if no secrets file exists at all - normal for local dev) and falls back to .env locally, so one codebase works in both places.

The retry safety net

invoke_with_retry() automatically retries a request up to twice, but only for a specific intermittent model glitch - any other error still fails immediately, so real problems are never hidden.

7. All Bugs & Issues Found and Fixed

This is the complete list - covering both bugs found in the original code, and real issues that only appeared after going live (model retirements, cloud-specific errors, deployment quirks). Every one below was confirmed by actually reproducing the failure before fixing it.

Issue 1: Diagram drawing could crash the whole request (main.py)

Problem: An unrelated external call (drawing a picture of the AI graph) shared the same error handling as the real travel-planning logic. If that picture-drawing service was unreachable, the ENTIRE request failed.

Fix: Isolated it in its own error handling so it can only log a warning, never block the real answer.

Issue 2: OpenAI model name was hardcoded (model_loader.py)

Problem: Always used a fixed model name, ignoring whatever was actually written in config.yaml.

Fix: Changed the code to use the model name read from config.yaml.

Issue 3: Broken currency conversion URL (currency_converter.py)

Problem: An extra slash in the web address could cause the request to fail or be misrouted.

Fix: Removed the extra slash so the address builds correctly.

Issue 4: App crashed instead of showing a clean error (streamlit_app.py)

Problem: Tried to 'raise' a plain text message as if it were an error - Python does not allow that, always crashing with a confusing `TypeError`.

Fix: Replaced with a proper on-screen error message instead of crashing.

Issue 5: Itinerary displayed as an unformatted grey block

Problem: Extra leading spaces on every line made Markdown treat the whole itinerary as plain code instead of formatted text.

Fix: Removed the extra leading spaces so it now displays with real headings and dividers.

Issue 6: Silent search failures (place_search_tool.py)

Problem: The backup search only ran if Google Places crashed - if it simply returned nothing useful, no backup was ever tried, leaving the AI with no data.

Fix: Restructured so the backup also runs whenever Google returns an empty result.

Issue 7: Typo in the environment variable template (.env.name)

Problem: Listed the Tavily key with two letters swapped - the real library only reads the correctly-spelled name, so the backup search would have been silently disabled.

Fix: Corrected the spelling in the template.

Issue 8: Whole agent crashed without a Google Places key

Problem: Building the agent tried to validate Google Places immediately, every request - one missing key blocked EVERY question, even ones that never needed place search.

Fix: Wrapped it in a safe check; missing/invalid key now falls back to Tavily-only instead of crashing.

Issue 9: Project secretly needed a paid API to run

Problem: Google Places requires a Google Cloud billing account (a card on file) even for its free credit - against the goal of running 100% free.

Fix: Rewrote the setup to mark Google Places and OpenAI clearly as optional; confirmed the app runs fully on just 4 genuinely free keys.

Issue 10: Groq retired the original model

Problem: The model this project was built on (deepseek-r1-distill-llama-70b) was fully decommissioned by Groq - not a code bug, an external dependency changing outside anyone's control.

Fix: Investigated Groq's current recommendations and confirmed a working replacement live on the user's own account, rather than guessing blindly.

Issue 11: Hotel cost calculator crashed on real numbers

Problem: A tool's input was typed as text instead of a number, so the AI's number arrived as text and multiplying text by a decimal number crashed with a TypeError.

Fix: Corrected the type, and re-tested with real numbers to confirm the fix.

Issue 12: Totaling trip costs together always crashed

Problem: Found while re-checking the fix above: a second tool used Python's variadic-argument syntax, which doesn't match how the AI actually calls it - crashing on almost every request that needed a total budget.

Fix: Rewrote the tool to accept a proper list argument matching how it's actually invoked.

Issue 13: A newer replacement model had a hidden formatting bug

Problem: The next model tried (openai/gpt-oss-120b) uses a different internal response format with documented, open issues in agentic tool-calling flows - including returning a completely blank final answer.

Fix: Switched to a model built specifically for reliable tool-calling, confirmed against real, current documentation rather than assumption.

Issue 14: A recommended model wasn't actually available on this account

Problem: The next model tried returned 'model not found / no access' - it exists on Groq generally, but wasn't enabled for this specific account.

Fix: Built a small script to query the account's own live model list directly from the Groq API, removing all further guesswork, and picked a confirmed-available, reliable model from that real list.

Issue 15: Long, complex responses could still glitch mid-answer

Problem: Even with a good model and a correct prompt instruction, a very long two-plan response could still occasionally end with a malformed inline tool-call attempt, rejected entirely by the API.

Fix: Added automatic retry logic at the code level (2 extra attempts) specifically for this one error type - unit-tested in isolation before integrating, confirming it retries only for this exact failure and never hides other real errors.

Issue 16: Local-only architecture broke on Streamlit Cloud

Problem: The Streamlit app called a separate FastAPI backend over localhost - Streamlit Community Cloud only runs one process, so that connection would never exist there.

Fix: Rewrote the frontend to call the LangGraph agent directly in the same process - works identically locally and in the cloud.

Issue 17: Checking cloud secrets crashed locally

Problem: Streamlit's `st.secrets` throws an error if no secrets file exists at all anywhere - normal for local .env-based development - instead of just returning nothing.

Fix: Wrapped the check in error handling so it's silently skipped locally, while still working correctly once deployed with real Cloud secrets.

Issue 18: Itinerary length didn't match the requested trip

Problem: The prompt's day-by-day example showed exactly 3 days as a formatting illustration - the AI treated that as a fixed rule and ignored longer requests like a 5-day trip.

Fix: Added an explicit instruction clarifying the example is a format template only, and the actual day count must always match what the user asked for.

Issue 19: A vague request (e.g. just 'plan a trip') crashed with a calculator error

Problem: With no destination or duration given, the AI still tried to proceed and call tools with missing or placeholder data, surfacing the old costs-argument bug again as a side effect - a UX gap as much as a code issue.

Fix: Added a guard at the very top of the prompt: if a destination or duration is missing, the AI now replies with a friendly clarifying question and calls no tools at all, instead of guessing.

8. Tools & Models Used (Step by Step Flow)

This diagram shows how the LLM and its tools are selected and connected today: config.yaml and ModelLoader pick llama-3.3-70b-versatile, that model is bound to four tool groups, and every response passes through an automatic retry safety net before reaching the user.

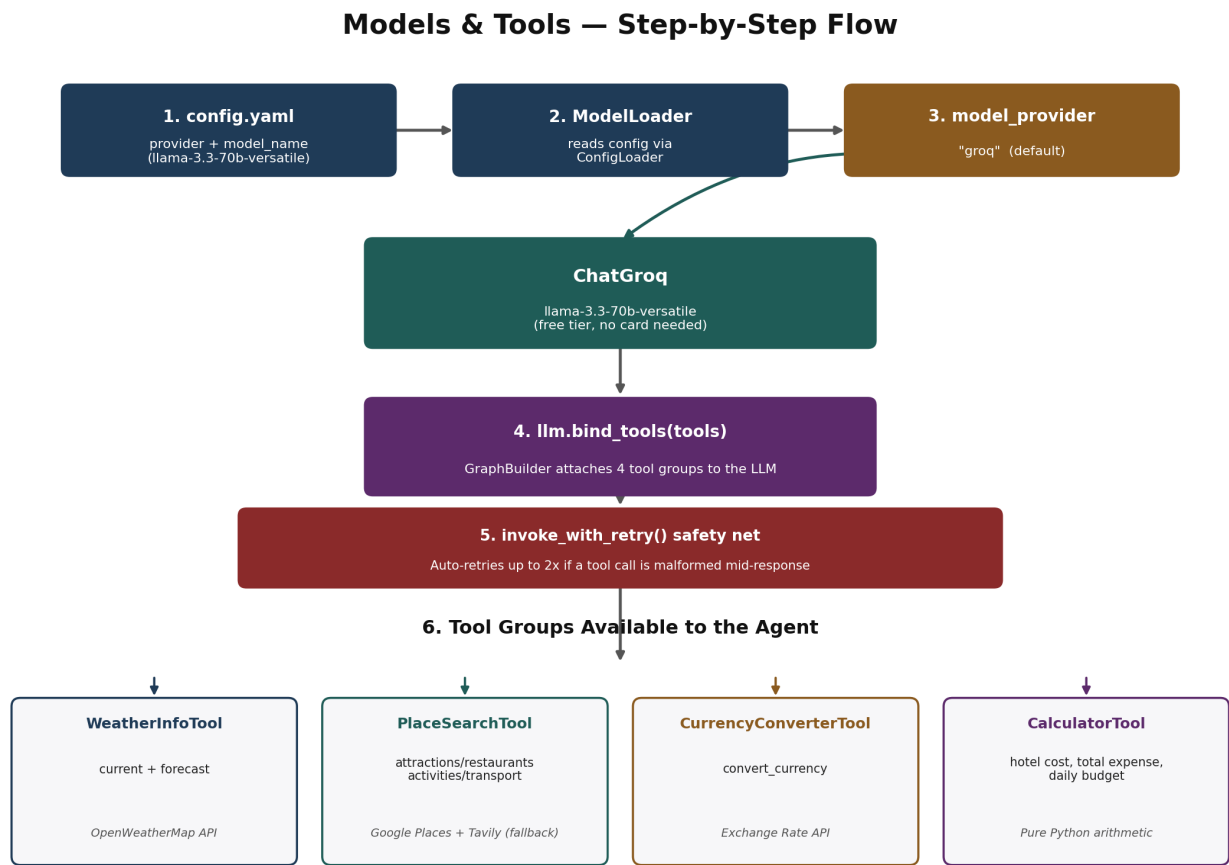


Figure 2: Step-by-step flow of model selection, tool binding, and the retry safety net.

9. Prompt Customizations Added

Beyond the original base prompt, several real customizations were added - each one purely additive, confirmed by diff against the previous version each time, so nothing already working was ever accidentally changed or removed along the way.

Two complete, separate plans

Every response now generates a full Plan A (popular/tourist) and Plan B (off-beat/less-crowded), each with its own complete itinerary, hotels, restaurants, activities, transportation, budget table, and packing suggestions - never mixed together.

Locations: name, description, then map link

Each place is shown as plain readable text with a 2-3 sentence description of what it is and why it's worth visiting, followed by a separate 'View on Map' Google Maps search link - not the place name itself turned into a link.

Budget as a Markdown table

The cost breakdown is always a proper table (Category / Amount / Notes) with a Total row, instead of a plain bullet list, for each plan separately.

Live weather vs. seasonal average

Weather always includes both the typical seasonal climate for the travel dates AND the actual live current weather, explicitly compared against that seasonal average.

Emoji-based visual structure

Section headings, categories, and key details use relevant emojis to make the long response easier to scan, without overusing them on every sentence.

Exact trip-length matching

The AI now generates exactly as many days as requested (5 days = Day 1 through Day 5), rather than defaulting to the 3-day structure shown as a formatting example.

Tool-call timing discipline

An explicit instruction requires the AI to gather ALL tool data first, before writing any final answer text - reducing (though, as the bug log shows, not fully eliminating) the risk of a malformed tool call appearing mid-response.

Graceful handling of vague or incomplete requests

Before doing anything else, the AI now checks whether the message includes both a destination and a duration. If either is missing (e.g. just "plan a trip"), it responds with a short, friendly clarifying question and calls no tools at all - instead of guessing and risking a crash from incomplete data.

10. Deployment: GitHub + Streamlit Community Cloud

The finished project is deployed as a real, live, public web app. This required both pushing the code to GitHub and configuring it correctly on Streamlit Community Cloud.

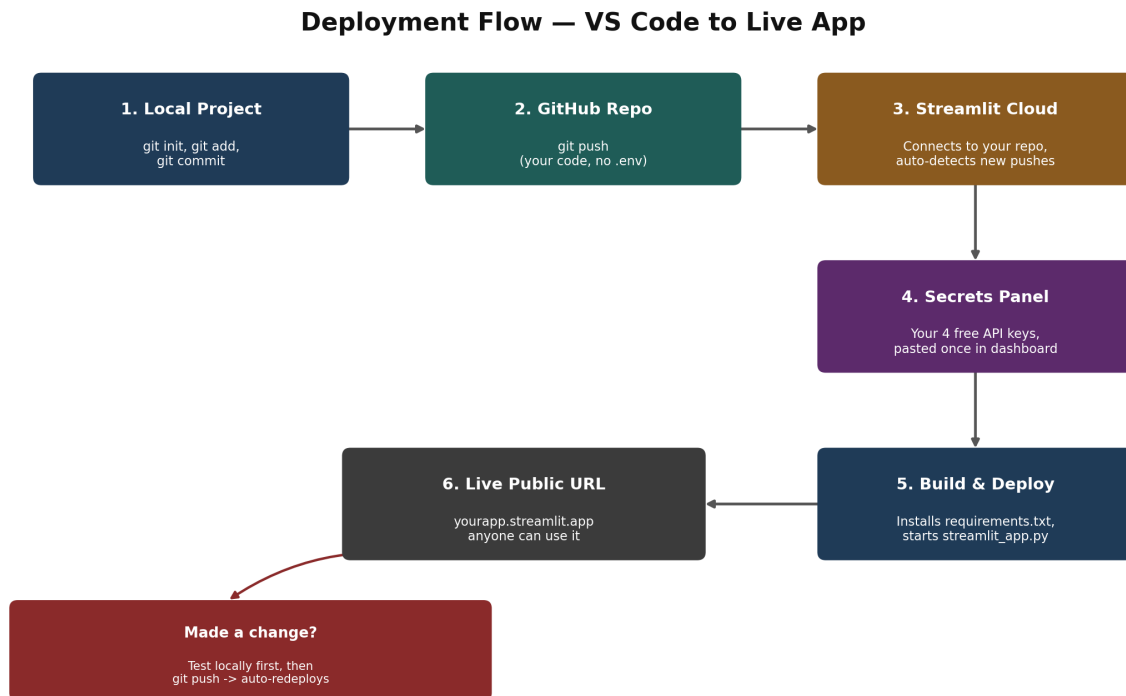


Figure 3: The full deployment pipeline, from local code to a live public URL.

Pushing to GitHub (VS Code terminal)

- `git init, git add ., git commit -m "message"` - stage and save changes locally
- `git remote add origin <repo-url>` - connect to a GitHub repository
- `git push -u origin main` - upload to GitHub
- A correct `.gitignore` (excluding `.env` and the virtual environment folder) is essential - never commit real API keys

Streamlit Community Cloud setup

- Sign in at share.streamlit.io with GitHub, click 'New app'
- Select the repository, branch, and set the main file to `streamlit_app.py`
- Add all API keys under Advanced Settings -> Secrets, in TOML format, before deploying
- Click Deploy - Streamlit Cloud installs `requirements.txt` and starts the app automatically
- Every future `git push` automatically triggers a redeploy - no manual step needed

Real problems hit during deployment (and how they were solved)

- **Push rejected (remote contains work not present locally):** happened when the GitHub repo wasn't truly empty. Fixed the first time with a force-push (safe, since nothing valuable was on the empty remote); the second time (real existing code) solved correctly with `git pull`, resolving the difference, then pushing normally.
- **Confusion about what git push actually does:** clarified that `git push` only uploads a copy to GitHub - it never touches, deletes, or moves anything on the local computer, which made the force-push decision safe to understand and make confidently.

- **Architecture didn't fit the cloud's single-process limit:** covered in the bug log above - solved by having Streamlit call the agent directly instead of a separate backend process.

Security incident: real API keys accidentally exposed

During setup, real API keys (Groq, Tavily, OpenWeatherMap, Exchange Rate) ended up typed directly into `.env.name` - the public template file meant to always stay empty - instead of the private, git-ignored `.env` file. This is a genuinely easy mistake to make, since both files look nearly identical.

What caught it: GitHub's push protection scanned the commit before it was accepted, recognized two of the four key formats, and blocked the push entirely with a clear GH013 error - meaning the keys were never actually made public, even though they were committed locally.

Full resolution, in order:

- Immediately rotated all four keys at their source (deleted the old key, generated a new one), even though only two were auto-detected - once real key values are visible anywhere, they should always be treated as compromised, not just the ones a scanner happens to catch
- Rewrote `.env.name` back to a clean, empty template
- Confirmed the bad commit had never actually reached GitHub (`git status` showed 'ahead of origin by 1 commit'), which meant it was safe to rewrite locally rather than needing a more invasive history-cleanup
- Used `git commit --amend` to replace the bad commit in place with the corrected file, then verified with `git show HEAD:.env.name` that no real key text remained anywhere in the commit before pushing

Why this matters: this is a real demonstration of defense in depth - a human mistake happened, but a platform-level safety net (GitHub's secret scanning) caught it before any actual damage, and it was resolved methodically: rotate first, verify the fix at the source-of-truth level (the commit itself, not just the working file), then push. This is a genuinely strong story for an interview.

11. How to Run the Project (Local & Live)

Option A - use the live deployed app

Simply open the public Streamlit Cloud URL in any browser. Nothing to install or start - it runs 24/7 on Streamlit's servers.

Option B - run locally (for testing changes)

- • Open the project folder in VS Code, open a terminal
- • Activate the virtual environment: `.\env\Scripts\Activate.ps1` (Windows PowerShell) - confirm the prompt shows (env)
- • Make sure the local .env file has real API keys
- • Run: `streamlit run streamlit_app.py` - opens automatically in the browser
- • No separate backend terminal is needed anymore for the chat UI to work

Publishing a change to the live app

- • Test the change locally first (Option B above)
- • `git add .` then `git commit -m "describe the change"`
- • `git push` - Streamlit Cloud detects it and redeploys automatically within a couple of minutes

Optional: the FastAPI backend

main.py still works as a standalone API for direct testing or a future separate frontend: run `uvicorn main:app --reload --port 8000` in its own terminal. This is entirely optional and not required for the deployed chat app.

12. Tips

- • Keep provider-specific logic behind one ModelLoader, so switching models only means editing config.yaml.
- • Always give an external-data tool a free backup, so one API outage or missing key does not break the whole agent.
- • Test each tool alone in a notebook before adding it to the agent.
- • Be very explicit in the system prompt about exact output structure - but understand prompt instructions alone have a reliability ceiling with open-weight models on very long generations.
- • Add code-level safety nets (like automatic retries for specific, known error types) rather than relying purely on prompt wording for reliability.
- • When a hosted model gets retired, check the current model list directly from your own account/API rather than guessing a replacement name.
- • Design for the most restrictive deployment target from the start where possible - a single-process architecture works both locally and on most free hosting platforms.
- • Never commit .env - always verify with 'git status' before pushing that secrets aren't included.
- • Understand what commands actually do before running them - e.g. git push only uploads a copy, it never touches or deletes local files.

13. Resume Notes

Resume / LinkedIn bullet point

AI Trip Planner using LLMs & LangGraph - Built and deployed an intelligent, tool-using travel assistant with LangChain, LangGraph, and Streamlit that generates dual (popular + off-beat) itineraries by dynamically calling real-time APIs for weather, places, and currency. Diagnosed and fixed 19+ real bugs across development and post-deployment, including model-provider migrations, cloud architecture constraints, and intermittent tool-calling failures; also identified and remediated an exposed-secret incident during deployment. Shipped live via GitHub and Streamlit Community Cloud.

One-line version

A deployed, agentic AI travel planner that turns a plain-language trip request into two full, real-data-backed itineraries - built, debugged through real production issues, and shipped live.

30-second spoken introduction

"I built and deployed an AI Trip Planner - a travel assistant that generates two complete itineraries, a popular one and an off-the-beaten-path one, using LangGraph to let the AI decide on its own when to check live weather, prices, and places. Along the way I worked through a lot of real issues you only hit in production - like a model provider retiring the model I was using, fixing tool-calling reliability with retry logic, and re-architecting the app so it would actually run on a cloud platform that only supports a single process. It's live now on Streamlit Community Cloud, running entirely on free-tier APIs."

14. Interview Talking Points

For any 'why did you choose X' question: state the alternative you considered, the concrete problem with it, then why your choice solves that problem. The deployment section below is a strong addition to your interview answers - it shows real debugging under real constraints.

Q: Why did you use an AI agent instead of a normal chatbot?

A: A normal chatbot only answers from memory, which goes stale. An agent calls live tools to fetch real data before answering, so the answer is actually correct.

Q: What was the biggest challenge in deploying this project?

A: The architecture assumed two always-running processes (a FastAPI backend and Streamlit talking over HTTP), but Streamlit Community Cloud only supports a single process. I rewrote the frontend to call the LangGraph agent directly in-process, which also simplified local development - no more running two terminals.

Q: Tell me about a real production issue you had to debug.

A: The LLM provider (Groq) retired the model my project depended on with no warning. I had to research the current recommended replacement, discovered that one had its own documented tool-calling bug causing empty responses, tried another that wasn't enabled for my account, and settled on a model with a proven tool-calling track record - confirmed against my own account's live model list rather than guessing.

Q: How do you handle unreliable AI responses in production?

A: I added a targeted retry mechanism that automatically retries a request up to twice, but only for one specific, identified error type (a malformed tool call). Any other error still fails immediately, so real problems are never silently hidden - I unit-tested this retry logic in isolation before wiring it into the app.

Q: Why does your place search use two providers?

A: For reliability, and so the whole project can run on free tiers only - Google Places requires billing even for its free credit, so I built an automatic fallback to Tavily instead.

Q: Have you ever had to deal with a security issue in a real project?

A: Yes - I accidentally typed real API keys into a public template file instead of my private, git-ignored one. GitHub's push protection caught it before it went public and blocked the push. I treated every affected key as compromised and rotated all of them immediately, not just the ones the scanner flagged, then confirmed with git commands that the secret was actually gone from the commit itself - not just the working file - before pushing again.

Q: What would you improve next?

A: Add automated tests for the tools, structured logging in production, and possibly separate the FastAPI backend onto its own free hosting service so it can be used independently of the Streamlit UI again.

End of guide - AI Trip Planner using LLMs & LangGraph. Built, debugged, and deployed.